

Validation Layer

Kernidee

Zero Trust auf Agent-generiertes Wissen. Jede Information die persistent wird, muss 6 Gates passieren. Denn falsches Wissen wird mit vollem Gewicht gespeichert, wenn man es nicht validiert. Temporal Decay filtert altes Wissen, aber falsches Wissen von gestern hat noch volles Gewicht.

Das Problem: Knowledge Corruption

Ein LLM das seine eigenen Aussagen prueft, bestaetigt sich selbst. "Ist das richtig?" → "Ja, das ist richtig." Das ist keine Validation, das ist eine Feedback-Schleife.

Echte Validation braucht externe Checks: Existiert die Datei? Stimmt der Fachbegriff? Widerspricht das dem bestehenden Wissen?

6 Validation Gates

Gate 1: Source-Pinning

Jeder Fakt bekommt eine nachpruefbare Quelle zugewiesen.

- `source=user://direct` → User hat es gesagt
- `source=file://pfad` → Aus einer Datei extrahiert
- `source=url://link` → Aus dem Web
- `source=agent://session` → Agent hat es generiert

Kein "laut verschiedenen Quellen". Konkret oder gar nicht.

Gate 2: Existence Check

Bevor eine Funktion, Datei oder API empfohlen wird: Existiert sie noch?

Simpler String-Match bzw. Dateisystem-Check. Kein LLM noetig. Verhindert Halluzinationen ueber gelöschte oder umbenannte Ressourcen.

Gate 3: Keyword Cross-Check

Stimmen die Fachbegriffe? Heisst es "Betriebsvereinbarung" oder "Betriebsverordnung"? Ein falscher Begriff in einem Governance-Dokument kann fatale Folgen haben.

Abgleich gegen bekannte Fachbegriffe im Hot Memory und Deep Storage.

Gate 4: Contradiction Check

Widerspricht die neue Information bestehenden Fakten?

Ablauf: 1. Agent will Fakt schreiben 2. Bestehende Fakten zum Thema abrufen 3. Vergleich: Widerspruch? 4. Wenn ja: NICHT ueberschreiben, sondern als Konflikt flaggen 5. User entscheidet welche Version gilt

Prinzip: User-Fakten haben immer Vorrang vor Agent-generierten Fakten.

Gate 5: Confidence Score

Drei Stufen: - **verified**: Primaerquelle, vom User bestaetigt - **unverified**: Agent-generiert, plausibel aber nicht geprueft - **rejected**: Widerlegt oder ueberholt

Niedrige Confidence = Review erforderlich. Nicht automatisch speichern.

Gate 6: Export-Filter

Beim Export aus Deep Storage in die Search Engine: - Notes mit `confidence=rejected` werden NICHT exportiert - Veraltete Eintraege werden markiert - Duplikate werden entfernt

Falsches Wissen schafft es nie in den Suchindex.

Gate 7: Cross-Model Validation

Dieselbe Frage an mehrere LLMs stellen und die Ergebnisse in einer frischen Sitzung vergleichen.

Das Problem das Gate 7 loest: Gates 1-6 pruefen Wissen gegen das eigene System — gegen bestehende Fakten, gegen das Dateisystem, gegen bekannte Begriffe. Aber ein LLM das sich selbst prueft, bestaetigt sich selbst. Gates 1-6 mindern das Problem, loesen es aber nicht vollstaendig: Pseudo-Statistiken (erfundene Zahlen die plausibel klingen), Kategorienfehler

(Metapher als Realitaet behauptet) und Barnum-Effekt (emotional validierende statt sachlich informierende Antworten) passieren alle 6 Gates, weil sie nicht im Widerspruch zu bestehenden Fakten stehen — sie sind eigenstaendige Fehler.

Architektur:



Synthese-Kriterien (definiert, nicht emergent):

1. **Faktenkonsens:** Behaupten alle Modelle dasselbe? Wenn 2 von 3 uebereinstimmen und eines abweicht, wird die Abweichung als Konflikt geloggt.
2. **Quellenpruefung:** Benennen die Modelle konkrete Quellen? Erfundene Zahlen haben keine Quelle — wenn kein Modell eine Studie benennen kann, ist die Zahl `confidence=rejected`.
3. **Tonanalyse:** Validiert die Antwort den User emotional oder informiert sie ihn sachlich? Divergenz zwischen Modellen in der Tonalitaet ist ein Signal fuer Barnum.
4. **Praezisionsvergleich:** Sagt Modell A "X ist Y" und Modell B "X verhaelt sich wie Y"? Die praezisere Formulierung gewinnt.
5. **Vollstaendigkeitspruefung:** Wie viele Dimensionen/Aspekte der Frage wurden beantwortet? Der Synthese-Agent zerlegt die User-Frage in ihre Teilaspekte und prueft fuer jedes Modell: Welche Aspekte wurden adressiert, welche ausgelassen? Die Synthese schliesst Luecken, indem sie fehlende Aspekte aus anderen Modellen ergaenzt. Ergebnis: Abdeckungsgrad pro Modell (z.B. "A: 4/6, B: 5/6, C: 3/6") und eine Synthese die alle Aspekte abdeckt.

Warum frische Sitzung fuer die Synthese: Der Synthese-Agent darf keinen Kontext aus den Einzel-Sitzungen haben. Sonst wird er vom ueberzeugendsten Output beeinflusst statt alle gleichwertig zu pruefen. Operative Geschlossenheit (Luhmann): Die Synthese operiert nur mit ihren eigenen Kriterien, nicht mit der Ueberzeugungskraft der Inputs.

Wann Gate 7 feuert: Nicht bei jeder Interaktion — das waere zu teuer (3x API-Kosten, Latenz).
Sondern: - Bei Faktenbehauptungen die persistent gespeichert werden sollen - Bei quantitativen Aussagen (Zahlen, Prozente, Statistiken) - Bei Aussagen ueber den User (Profil, Faehigkeiten, Muster) - On-Demand durch den User

Implementierungshinweis: Technisch realisierbar ueber parallele API-Calls an verschiedene Anbieter. Die Synthese laeuft in Claude Code (als primaeres System), aber in einer isolierten Session ohne Kontext-Bleed. Ergebnis wird mit `source=cross-validated` getaggt — eine neue Source-Kategorie oberhalb von `raw`.

Offene Designfragen: - Welche Modelle bilden das Minimum Viable Panel? (2 reichen fuer Widerspruchserkennung, 3 fuer Mehrheitsentscheid) - Wie werden die Synthese-Kriterien gewichtet? (Faktenkonsens > Praezision > Ton?) - Soll der User das Panel konfigurieren koennen? - Kosten-Management: Welche Fragen rechtfertigen 3x API-Kosten?

Zusammenspiel mit anderen Komponenten

- **Temporal Decay** filtert altes Wissen (zeitbasiert)
- **Validation Layer** filtert falsches Wissen (qualitaetsbasiert)
- **3-von-3 Quality Gate** prueft Output vor Auslieferung an den User
- Zusammen: Dreifache Absicherung gegen Knowledge Corruption

Erstellt: 24.03.2026 | Autor: Holger Woelfle